

but we then directly change the attribute's value to 'changed value'. When you run this program, the output looks like this:

```
some initial value
changed value
```

This output indicates that code can easily change `someAttribute` to any value. The downside of using regular attributes is that your code can set the `someAttribute` attribute to invalid values. This flexibility is simple and convenient, but it also means `someAttribute` could be set to some invalid value that causes bugs.

Let's rewrite this class using properties by following these steps to do this for an attribute named `someAttribute`:

1. Rename the attribute with an underscore prefix: `_someAttribute`.
2. Create a method named `someAttribute` with the `@property` decorator. This getter method has the `self` parameter that all methods have.
3. Create another method named `someAttribute` with the `@someAttribute.setter` decorator. This setter method has parameters named `self` and `value`.
4. Create another method named `someAttribute` with the `@someAttribute.deleter` decorator. This deleter method has the `self` parameter that all methods have.

Open a new file editor window and enter the following code, saving it as *propertiesExample.py*:

```
class ClassWithProperties:
    def __init__(self):
        self.someAttribute = 'some initial value'

    @property
    def someAttribute(self): # This is the "getter" method.
        return self._someAttribute

    @someAttribute.setter
    def someAttribute(self, value): # This is the "setter" method.
        self._someAttribute = value
```